File System Interface

File Concept

- The file system is the most visible aspect of an operating system.
- The file system consists of two distinct parts:
  - a collection of files, each storing related data
  - a directory structure, which organizes and provides information about all the files in the system.
- A file is a named collection of related information that is recorded on secondary storage.

File Concept…

- Types:
  - Data
    - numeric
    - character
    - binary
  - Program
- Contents defined by file's creator
  - Many types
    - Consider **text file, source file, executable file**

File Attributes

- **Name** – only information kept in human-readable form.
- **Identifier** – a unique tag (i.e., an internal number) that identifies the file within the file system.
- **Type** – needed for systems that support different types.
- **Location** – a pointer to file location on device.
- **Size** – current file size.
- **Protection** – controls who can do reading, writing, executing.
- **Time, date, and user identification** – data for protection, security, and usage monitoring.
- Information about files are kept in the directory structure, which is maintained on the disk

File Operations

- Creating a file (create)
- Writing a file (write)
- Reading a file (read)
- Repositioning within a file (seek)
- Deleting a file (delete)
- Truncating a file. (with empty content)

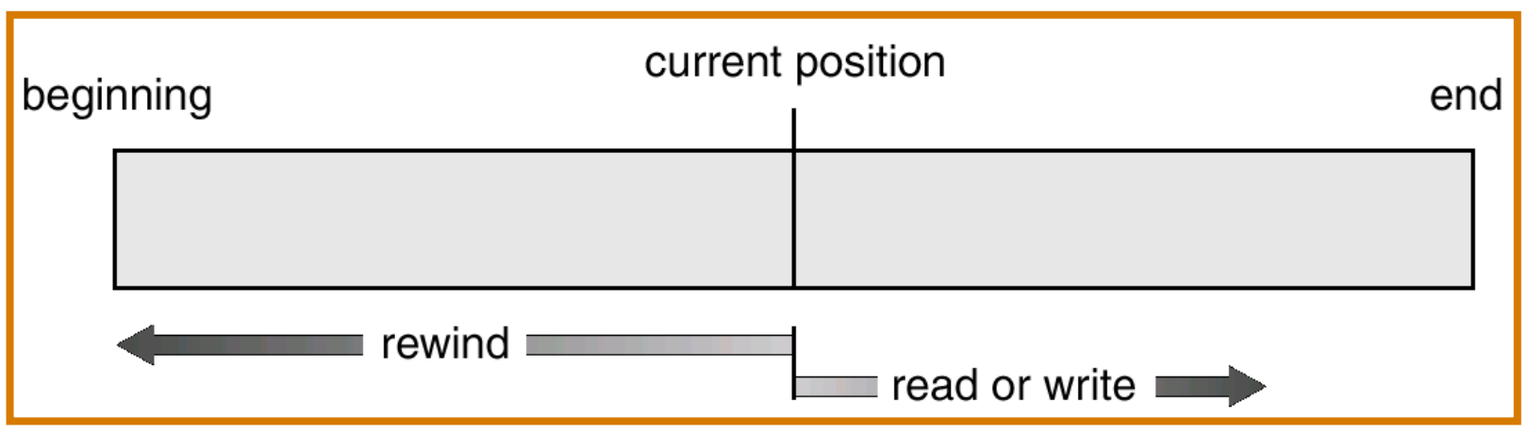
File Types

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

Figure 10.2 Common file types.

Access Methods

- Sequential Access
  - Information in the file is processed in order, one record after other.
  - Operations
    - Read next: reads the next portion of the file and automatically advances a file pointer.
    - Write next: write a record and advance the tape to the next position.
    - Few also support rewind/forward to skip n records perhaps n= 1.
- Best for data storage devices that read data sequentially(ex: Magnetic tapes)



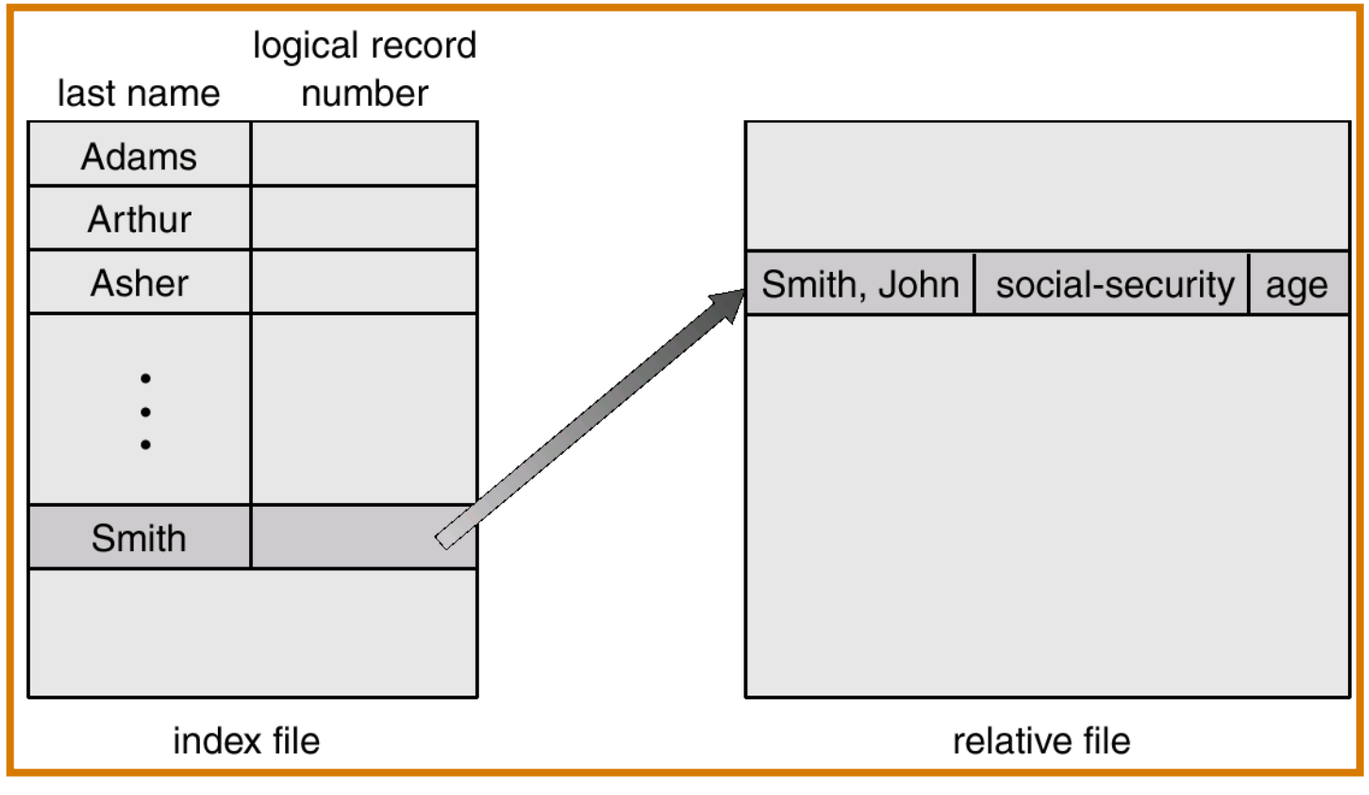
Access Methods…

- Direct Access/Relative Access
  - Jump to any record and read that record.
  - Operations
    - read n - read record number n
    - write n - write record number n.
    - jump to record n - could be 0 or the end of file.
  - Sequential access can be easily emulated using direct access. ex: for read n we would position to n and then read next.
  - n is normally relative block number. i.e index relative to the beginning of the file.

sequential access	implementation for direct access
reset	*cp* = 0;
read next	*read cp*; *cp* = *cp*+1;
write next	*write cp*; *cp* = *cp*+1;

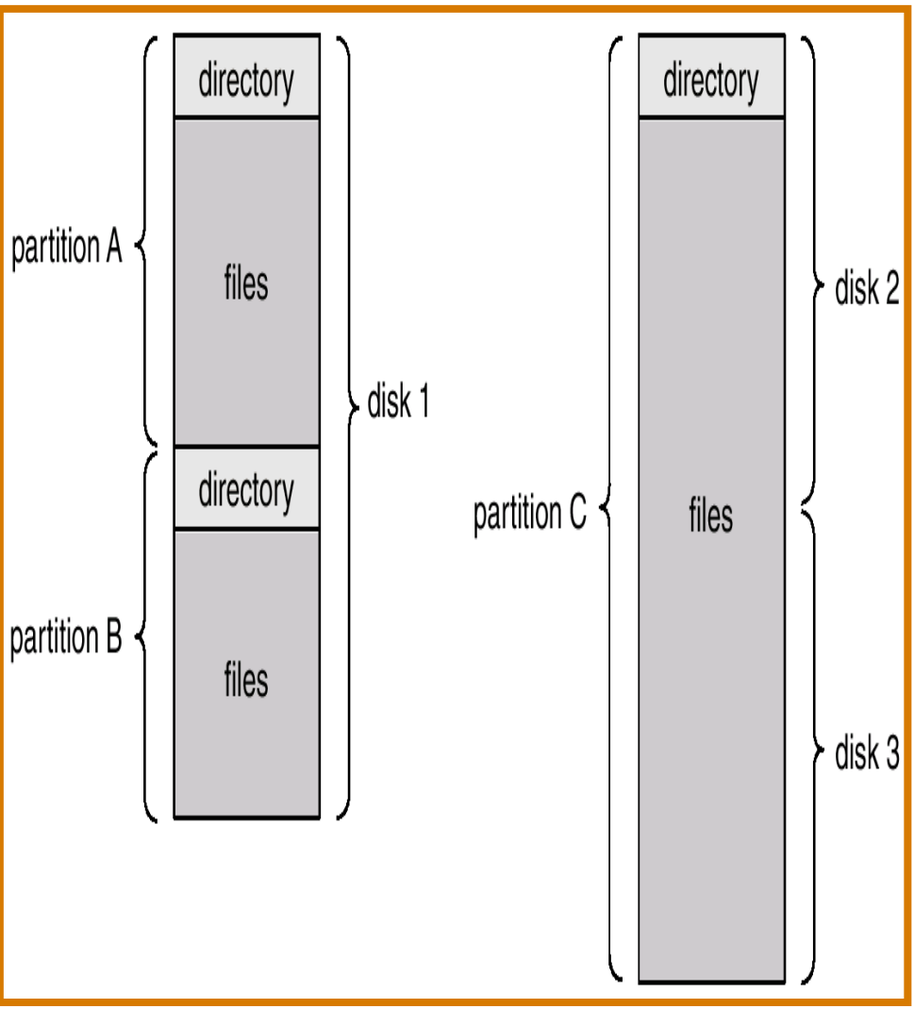
Other Access Methods

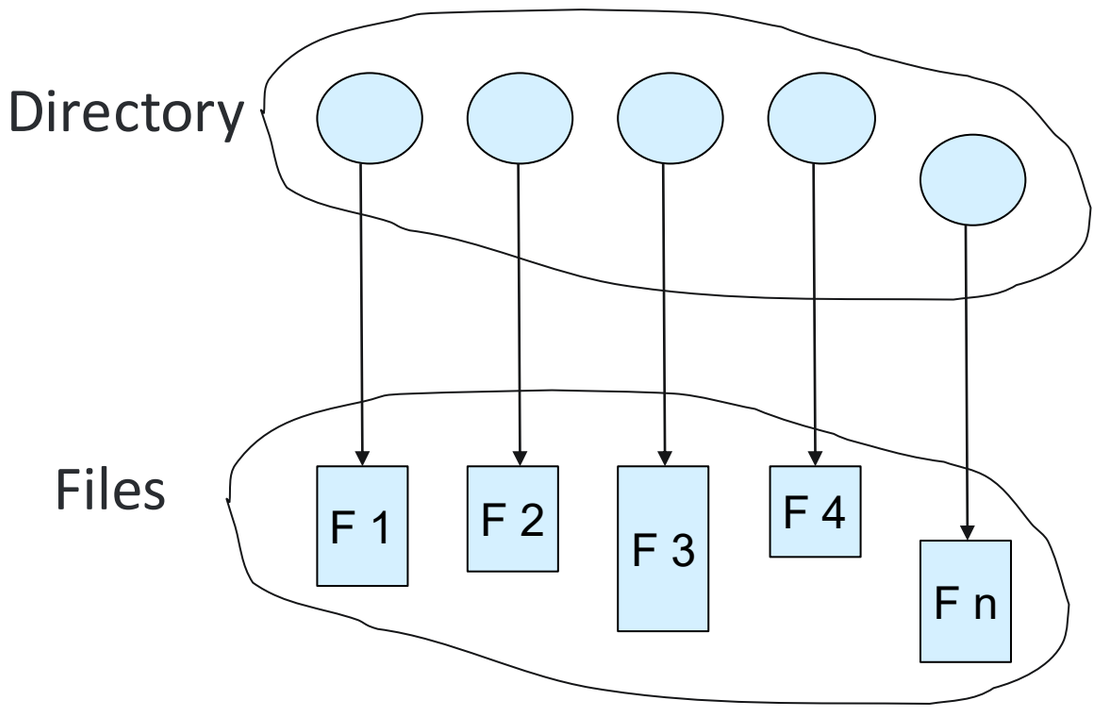
- An indexed access scheme can be easily built on top of a direct access method. Very large files may require a multi-tiered indexing scheme, i.e. indexes of indexes. ex: ISAM (indexed sequential-access method)



Directory Structure

A collection of nodes containing information about all files.





Both the directory structure and the files reside on disk.

Operations Performed on Directory

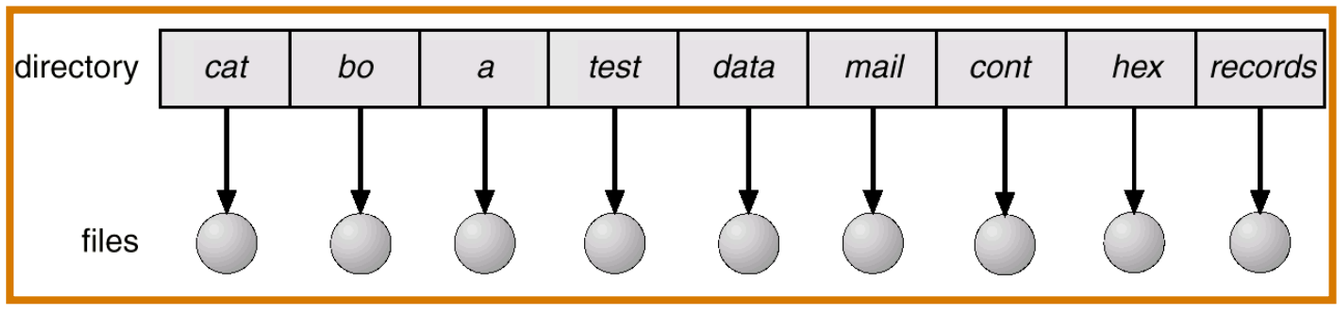
- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

Directory Structure…

- Organize the Directory (Logically) to Obtain
- **Efficiency** – locating a file quickly.
- **Naming** – convenient to users.
  - Two users can have same name for different files.
  - The same file can have several different names.
- **Grouping** – logical grouping of files by properties, (e.g., all Java programs, all games, …)

Single-Level Directory

A single directory for all users.

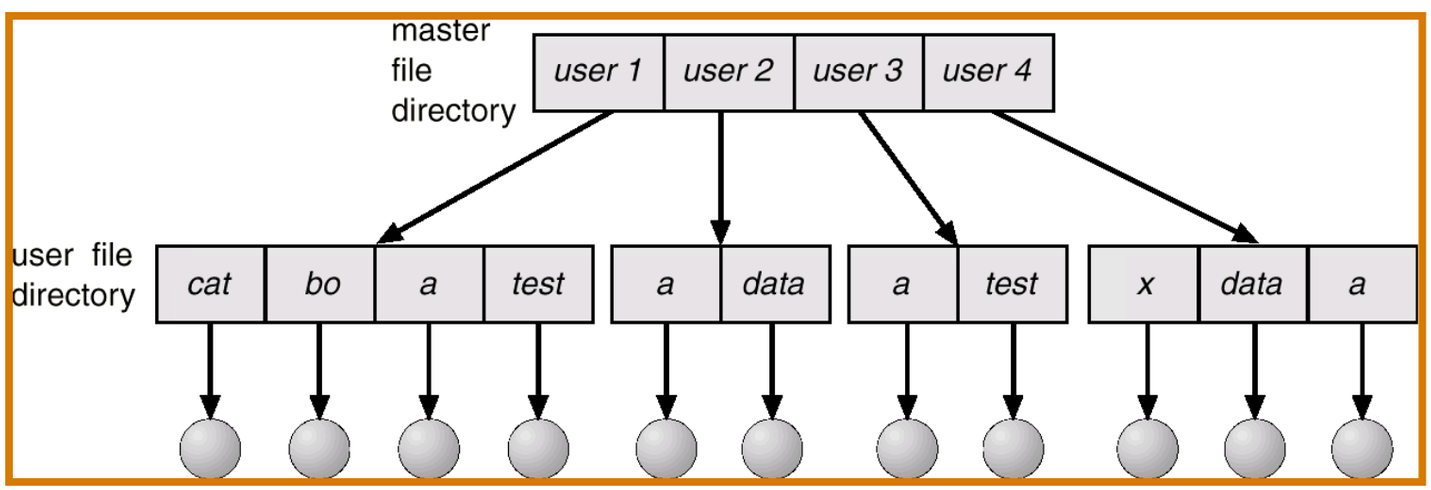


Naming problem

Grouping problem

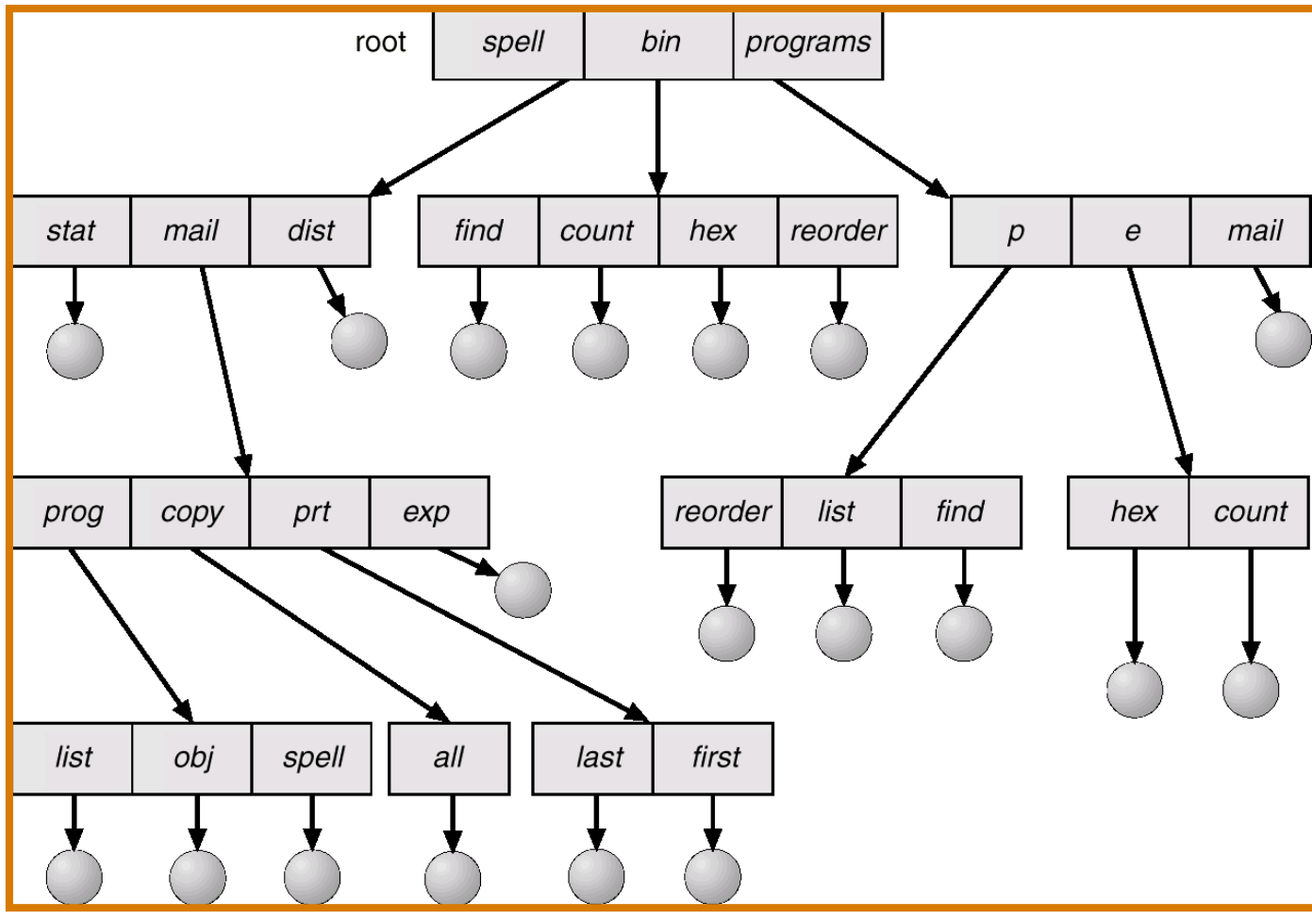
Two-Level Directory

- Introduced to remove naming problem between users
  - First Level contains list of user directories
  - Second Level contains user files
  - Need to specify Path name
  - Can have same file names for different users.
  - System files kept in separate directory or Level 1.
  - Efficient searching



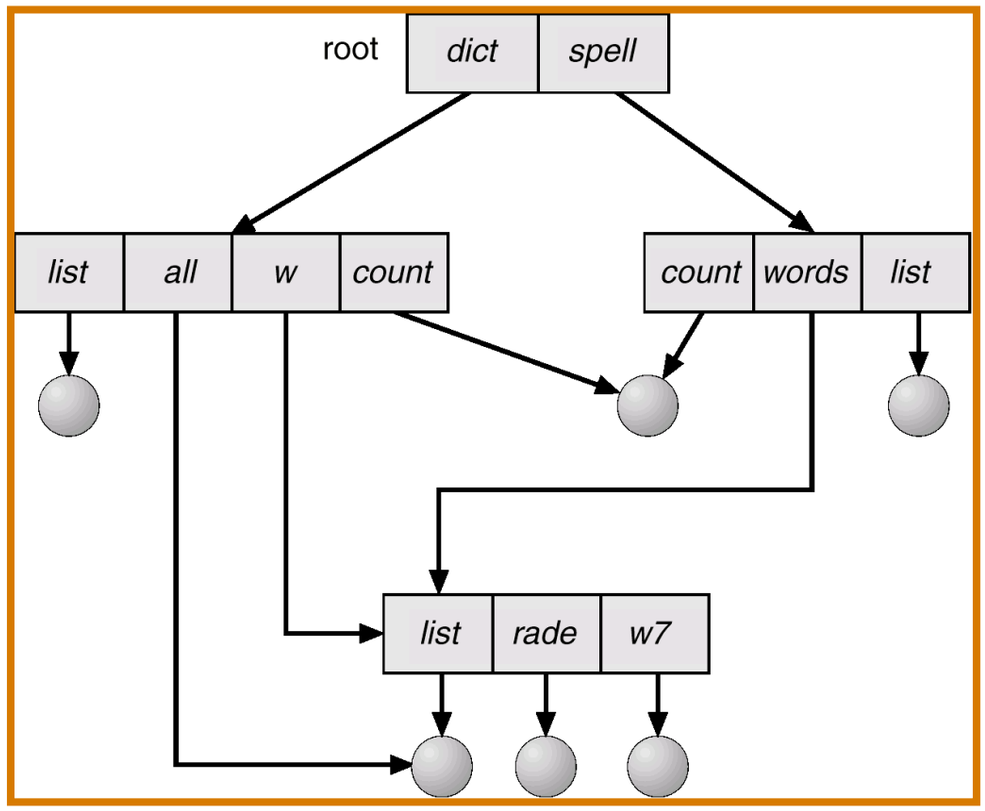
Tree-Structured Directories

- Arbitrary depth of directories
  - Leaf nodes are files, interior directories.
- Efficient Searching
- Grouping Capability
- Current Directory (working directory)
  - cd /spell/mail/prog
  - type list
- MS-DOS uses a tree structured directory
  - Absolute or relative path name
    - Absolute from root
    - Relative paths from current working directory pointer.

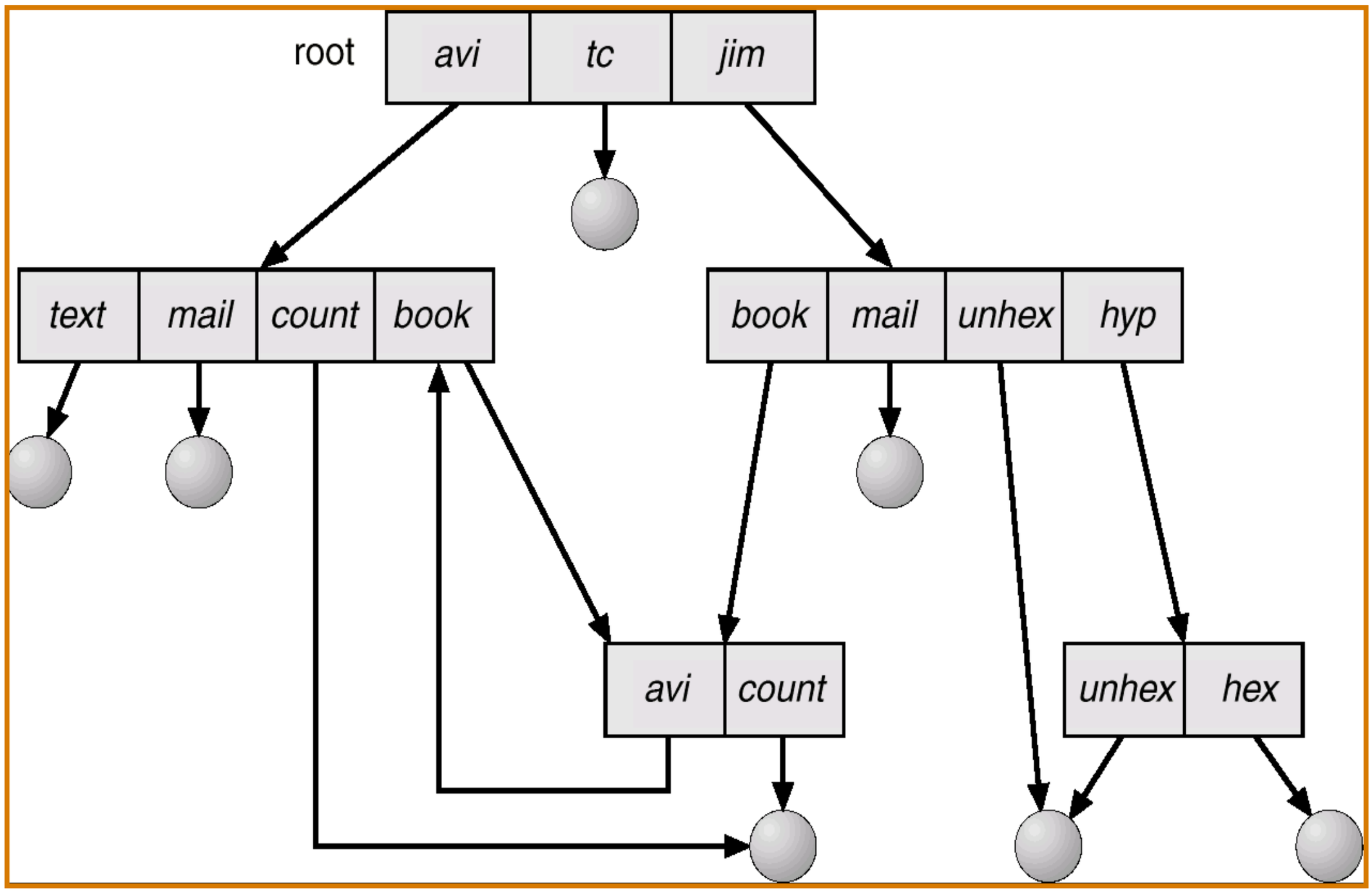


Acyclic-Graph Directories

- Acyclic graphs allow sharing
- Implementation by links
  - Links are pointers to other files or subdirectories
  - Symbolic links or relative path name
    - Directory entry is marked as a link and name of real file/directory is given. Need to resolve link to locate file.
- Implementation by shared files
  - Duplicate information in sharing directories
  - Original and copy indistinguishable.
  - Need to maintain consistency if one of them is modified.



General Graph Directory

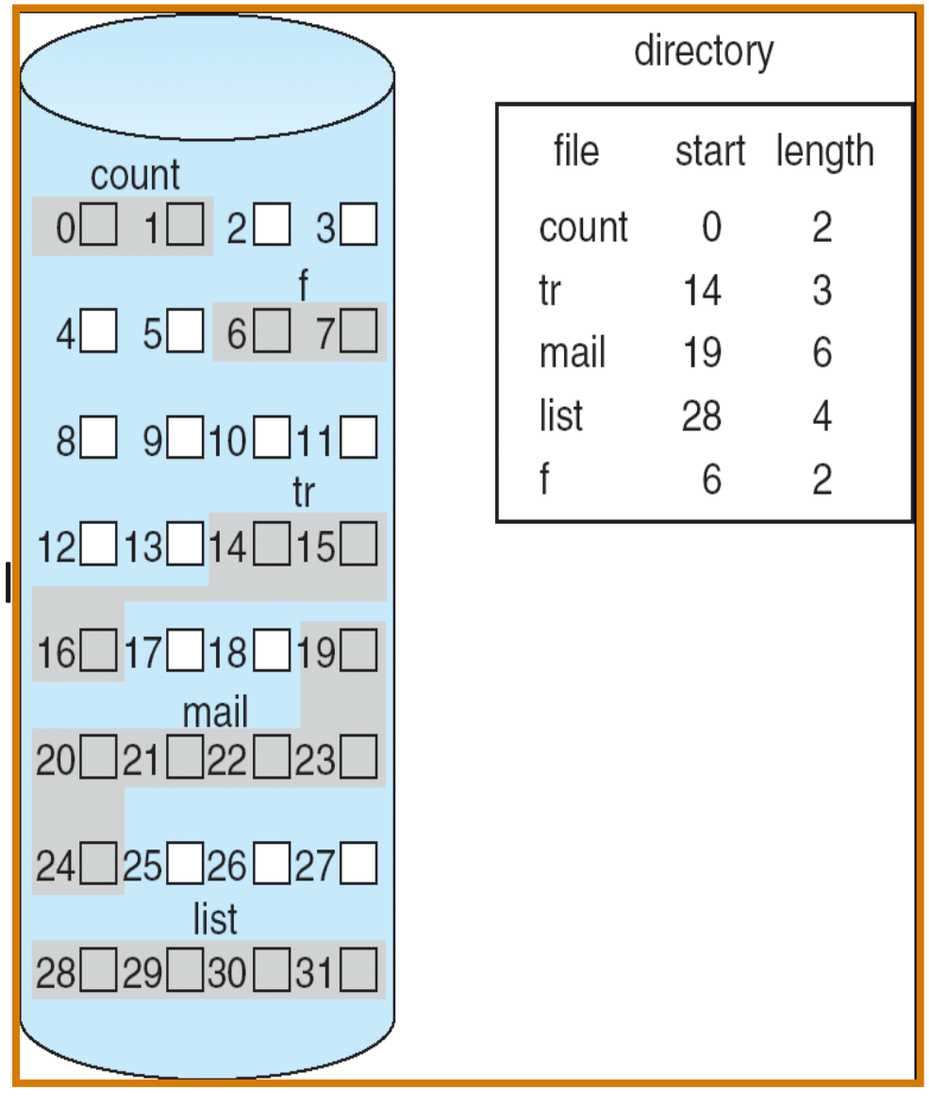


Allocation Methods

- Allocation methods deal with allocation of space to files so that the disk space is utilized effectively and files can be access quickly.
- Three major methods of allocating the disk space are in wide use
  - Contiguous allocation
  - Linked allocation
  - Indexed allocation

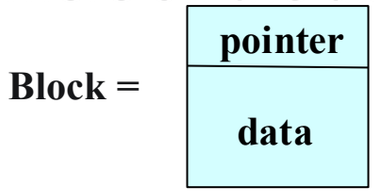
Contiguous Allocation

- Each file occupies a set of contiguous blocks on the disk.
- Simple allocation method. Only starting location (block#) and length (number of blocks) are required.
  - For a file *n* blocks long and starts at location *b*, then it occupies blocks *b, b+1, b+2, …, b+n-1*
  - The directory entry for each block includes the starting address of each block and the length allocated for this fil
- Contiguous allocation has some problems
  - Dynamic storage-allocation
  - External fragmentation
  - Determining how much space is needed for a file
    - Allocate too little and the file may not be extended
    - Allocate too much and space is wasted
  - To minimize these drawbacks, some operating systems use a modified version -> initially a contiguous chunk space is allocated and then, when that amount is not large enough, another chunk, an extent is added to initial allocation.

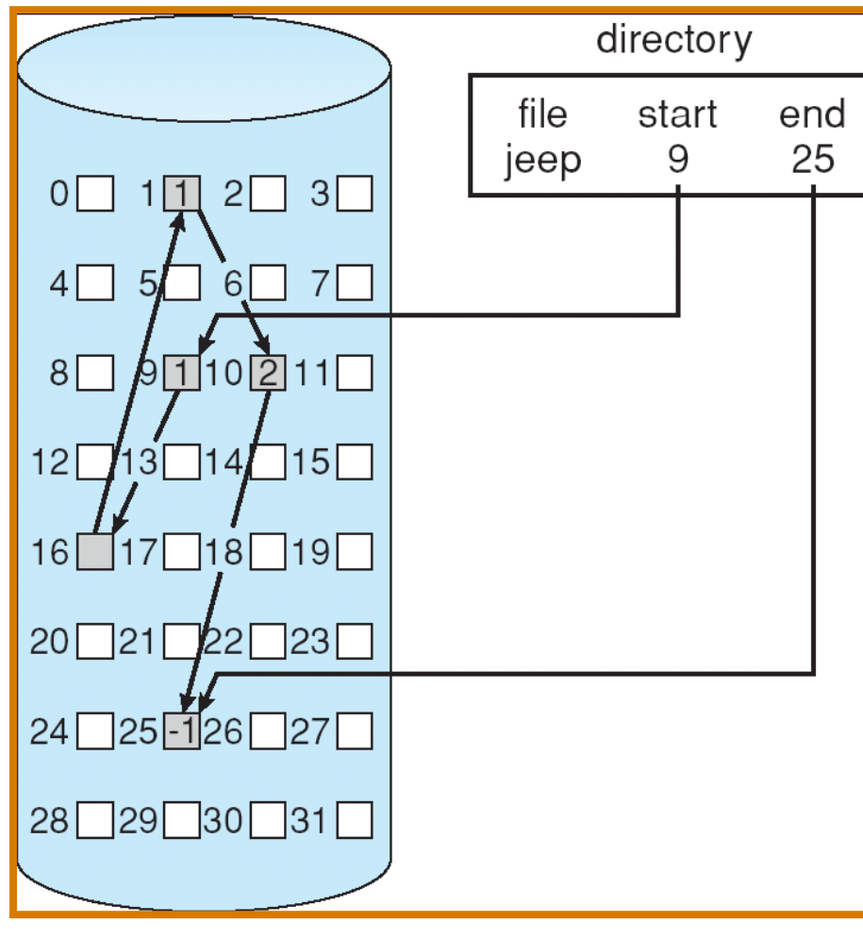


Linked Allocation

- Solves all the problems of contiguous allocation
- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk

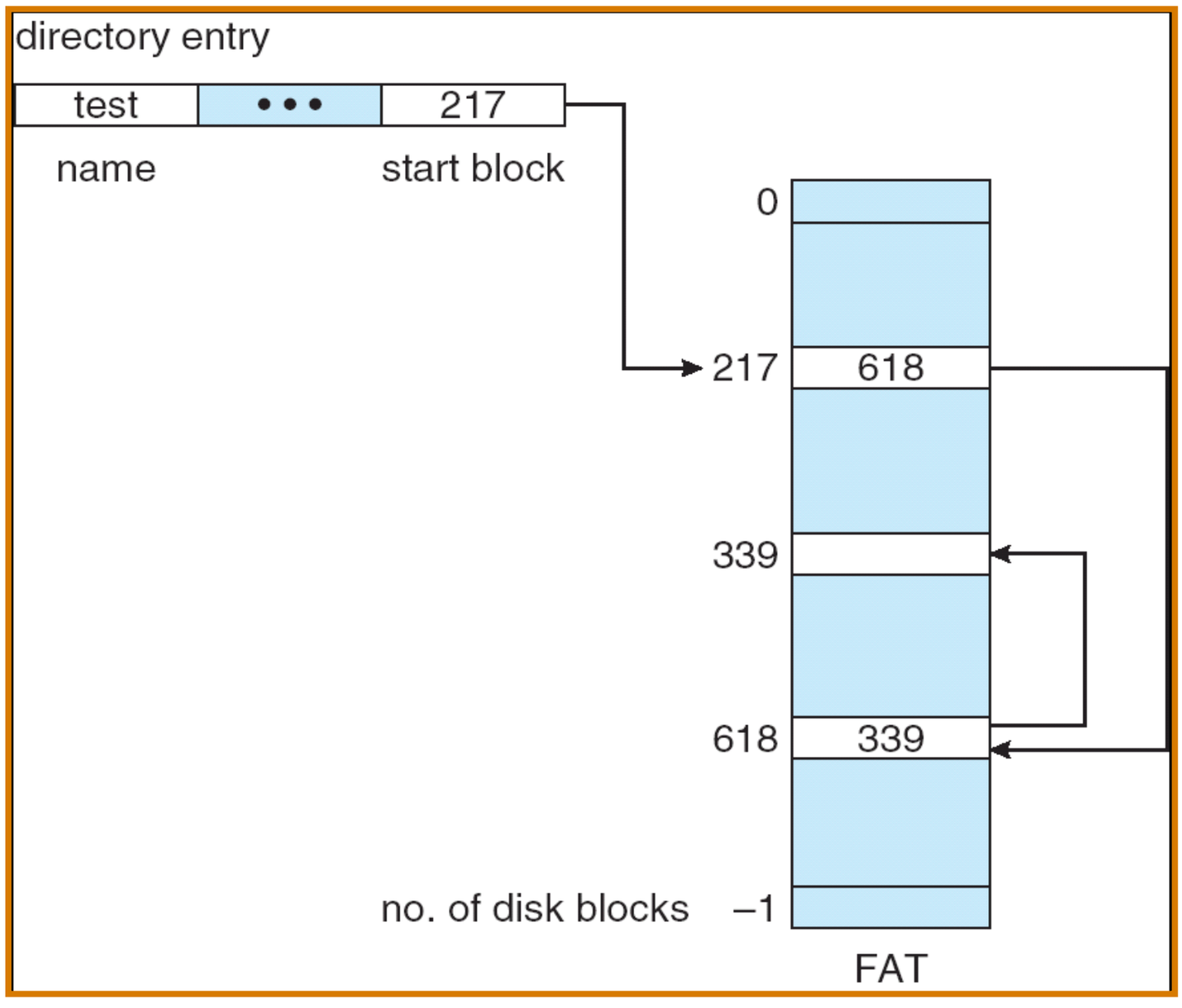


- The directory contains a pointer to the first and last blocks of a file.
- No external fragmentation
- Any free block on the free-space list can be used to satisfy a request
- A file can grow as long as free blocks are available, never need to compact disk space
- Linked allocation does have disadvantages
  - Only effective for sequential-access files
  - Space required for the list pointers. Use clusters to improve disk usage and access time.
  - Reliability



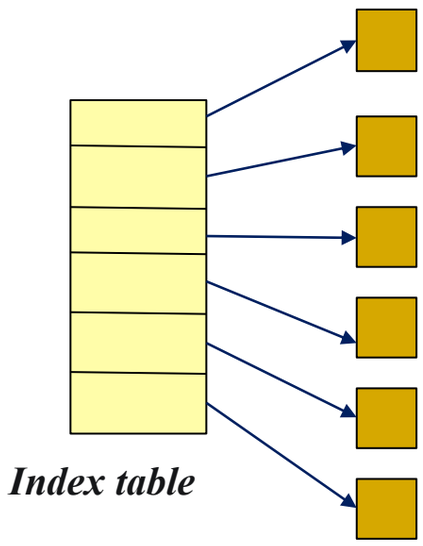
- The File Allocation Table (FAT) is a variation to the linked allocation method used to support direct access

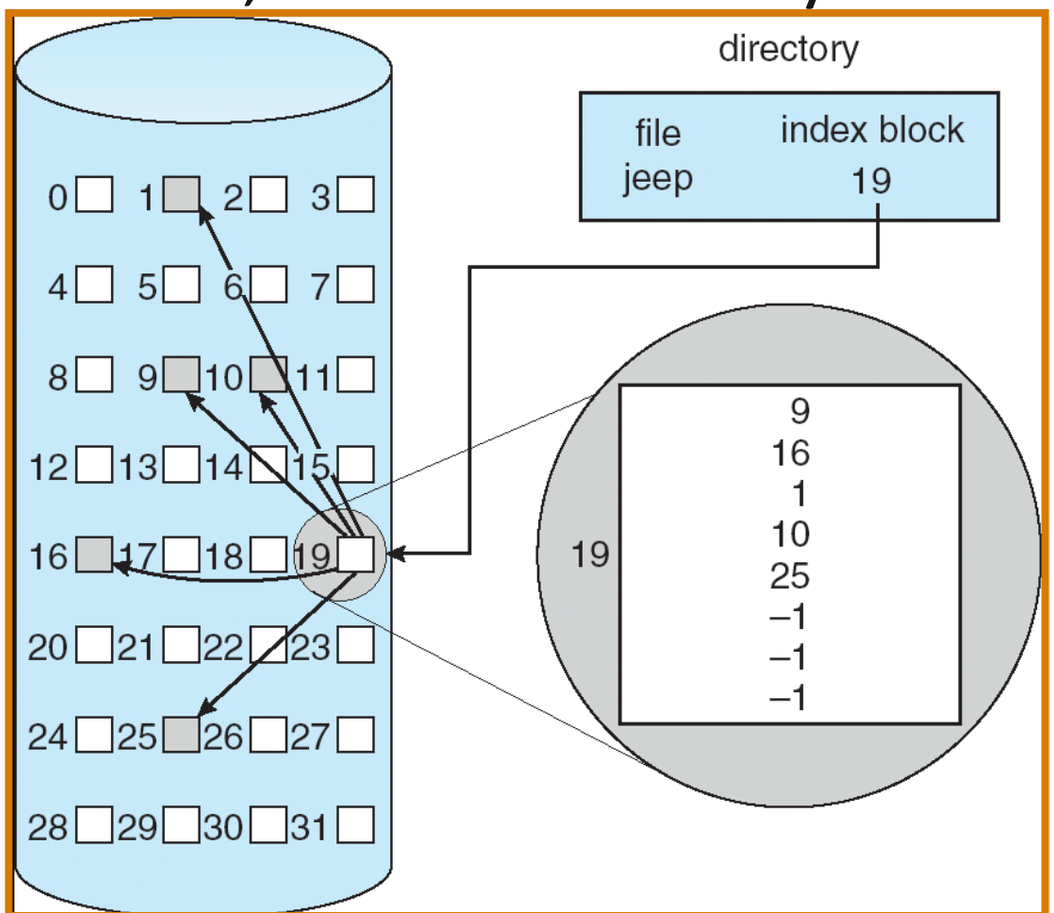
File Allocation Table



Indexed Allocation

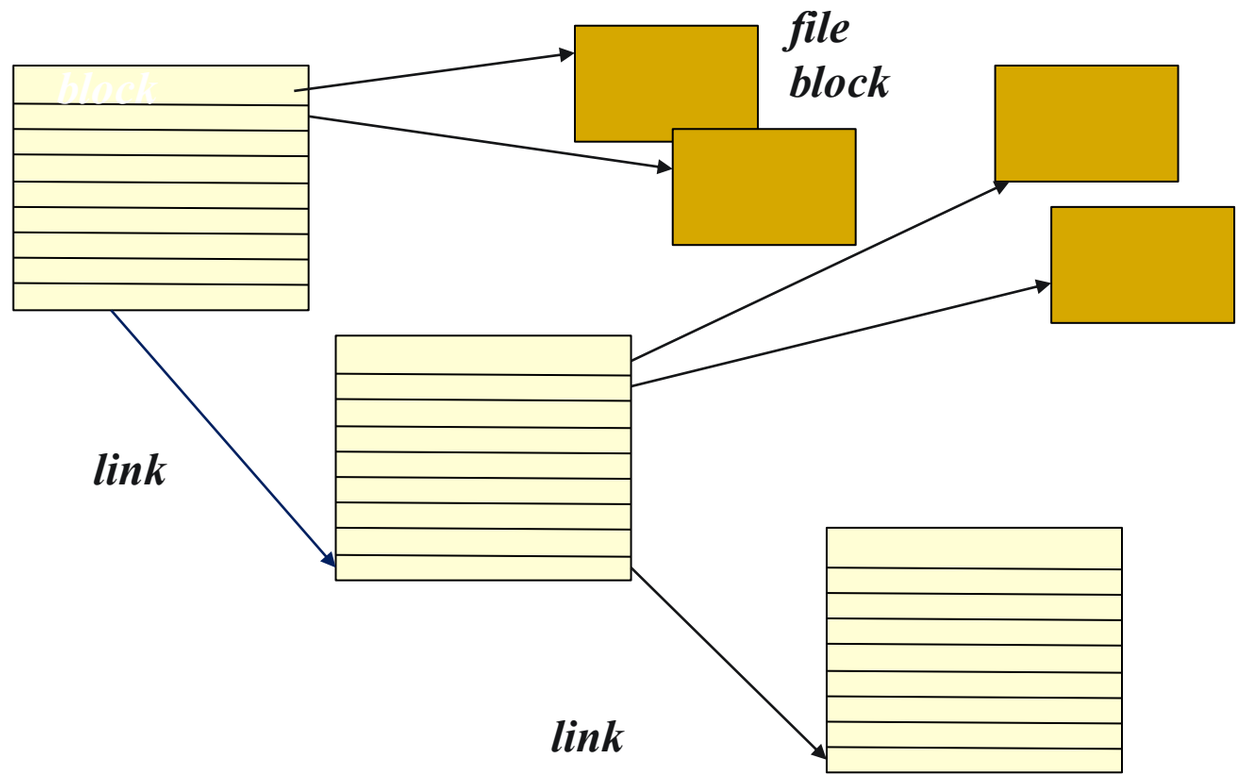
- Solves the external-fragmentation and size-declaration problems of contiguous allocation
- Supports direct access by bringing all the pointers together into the index block
- Each file has its own index block, which is an array of disk-block addresses





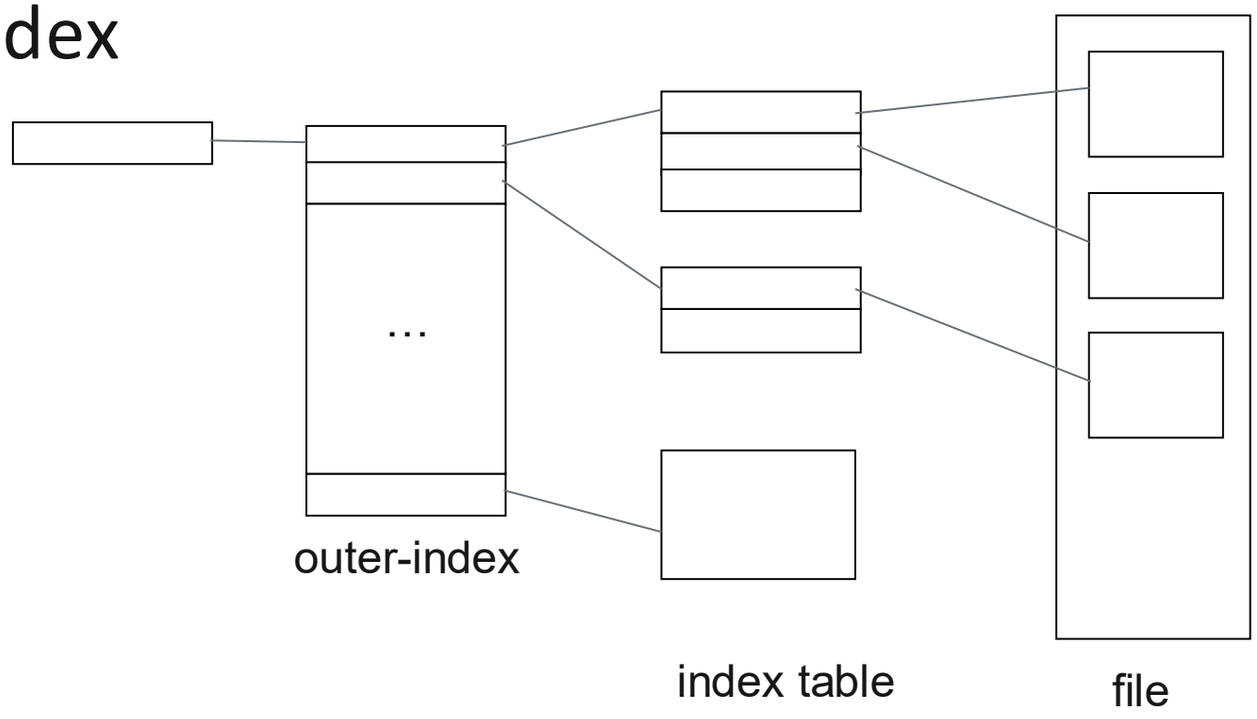
Indexed Allocation…

- Indexed allocation suffer from wasted space. If the index block is too small, it will not be able to hold the enough pointers for a large file.
- Indexed block can be handled in different ways.
  - Linked Scheme

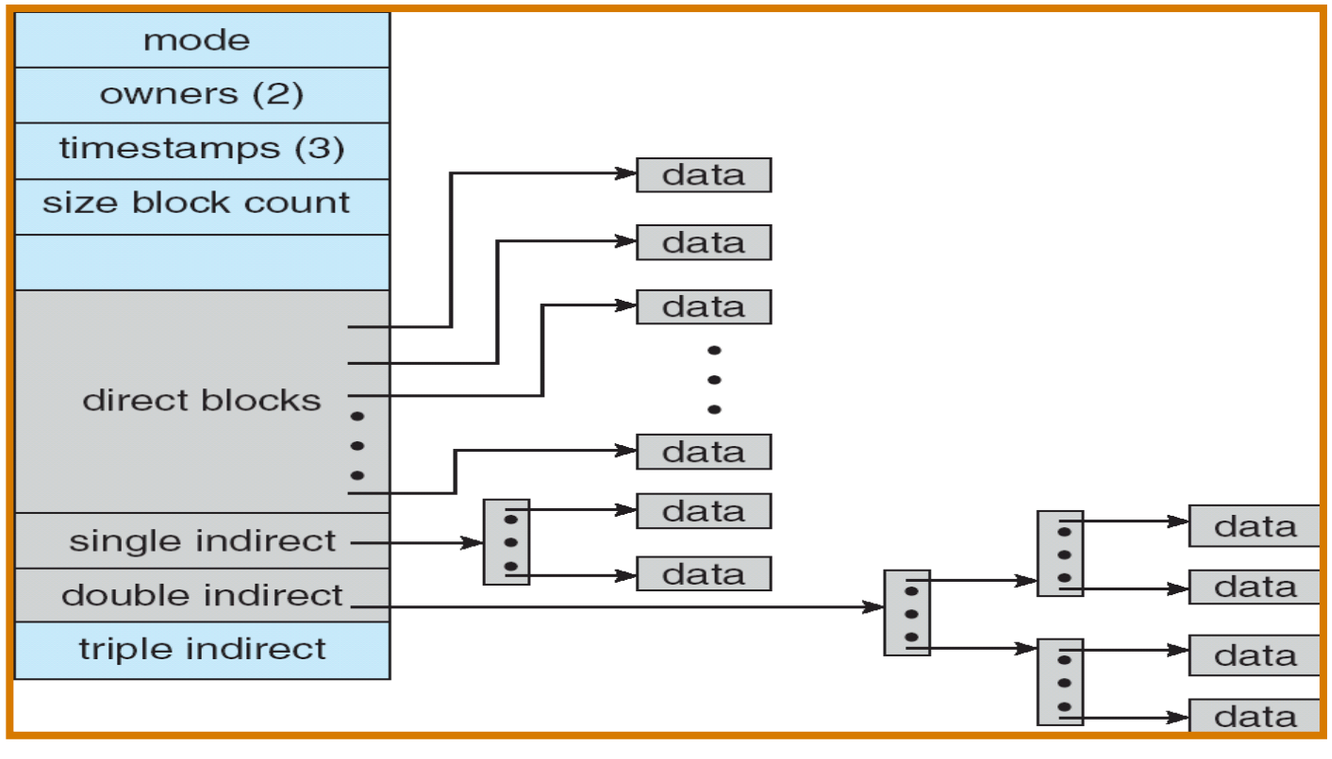


Indexed Allocation…

- Multilevel index



- Combined Scheme



Free Space Management

- Need to reuse the space from deleted files for new files
- To keep track of free disk space, the system maintains a free-space list
  - Stores all free blocks – those not allocated to a file or directory
- To create a file the free-space list is searched and that space is allocated to the new file, this space is then removed form the list
- When a file is deleted its disk space is added to the free space list

Bit Vector

- Frequently, the free-space list is implemented as a bit-map or bit vector
  - Each block is represented by 1 bit
  - If the block is free; the bit is 1; if the block is allocated the bit is 0

- Easy to get first free block or n consecutive free blocks on the disk.
- Bit Vector requires extra space

  Eg. Block size = 2^{12} bytes, Disk size = 2^{30} bytes

  n = $2^{30}/2^{12}$ = 2^{18} bits (or 32K bytes)

Free Space Management…

- Linked list (free list)
  - Keep a linked list of free blocks
  - Cannot get contiguous space easily, not very efficient because linked list needs traversal.
- Grouping
  - Keep a linked list of index blocks. Each index block contains addresses of free blocks and a pointer to the next index block.
  - Can find a large number of free blocks contiguously.
- Counting
  - Linked list of contiguous blocks that are free
  - Free list node contains pointer and number of free blocks starting from that address.
  - The Overall list will be shorter.